

Java: Cadeias de caracteres e arquivos

String • StringBuffer • File • FileWriter • FileReader

Prof. Dionizio Porfírio
Unichristus • 2026.2

Objetivos de aprendizagem

Ao final da aula, você deverá ser capaz de:

- armazenar e manipular cadeias de caracteres com String e StringBuffer;
- acessar, comparar, recortar e transformar textos;
- percorrer uma cadeia caractere por caractere;
- representar um arquivo por meio da classe File;
- gravar e ler caracteres em arquivos de texto;
- tratar IOException com try-catch;
- combinar Strings e arquivos em um cadastro simples persistente.

Cadeias de caracteres em Java

Cap. 9

Em Java, cadeias de caracteres podem ser armazenadas em objetos das classes `String` e `StringBuffer`. Cadeias ficam entre aspas ("texto"); caracteres isolados ficam entre apóstrofes ('A'). A manipulação básica dessas classes pertence ao pacote `java.lang`, que não precisa ser importado.

EXEMPLOS

```
String nome = "Maria";  
char letra = 'M';  
StringBuffer texto = new StringBuffer("Java");
```

`String` é a escolha comum para textos. Quando você precisa inserir, apagar ou alterar caracteres diretamente, o utilize `StringBuffer`.

Inicialização e entrada pelo teclado

ATRIBUIÇÃO

```
String curso = "Computação";  
String vazio = new String();
```

TECLADO

```
Scanner entrada = new Scanner(System.in);  
String nome = entrada.nextLine();
```

next() × nextLine()

next()

Lê até o primeiro espaço em branco. Ao digitar “Maria da Silva”, guarda apenas “Maria”.

nextLine()

Lê todos os símbolos digitados até ENTER. É a opção mais útil para nomes completos e frases.

Operações essenciais com String

Objetivo	Exemplo	Resultado / ideia
Tamanho	texto.length()	quantidade de caracteres
Caractere	texto.charAt(i)	caractere da posição i
Recorte	texto.substring(2, 5)	posições 2 até 4
Concatenar	a + b / a.concat(b)	junta duas cadeias
Comparar	a.equals(b)	true se forem iguais
Ignorar caixa	a.equalsIgnoreCase(b)	ignora maiúsculas/minúsculas
Localizar	texto.indexOf("Java")	posição ou -1
Limpar bordas	texto.trim()	remove espaços do início/fim
Transformar	toUpperCase() / toLowerCase()	muda caixa

Exemplo: normalizando e consultando um texto

```
import java.util.Scanner;

public class ExemploString {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        System.out.print("Digite seu nome: ");
        String nome = entrada.nextLine().trim();

        System.out.println("Tamanho: " + nome.length());
        System.out.println("Maiúsculas: " + nome.toUpperCase());
        System.out.println("Primeira letra: " + nome.charAt(0));
        System.out.println("Possui Silva? " + nome.contains("Silva"));
    }
}
```

Observe

1. `nextLine()` lê o nome completo.
2. `trim()` retira espaços nas extremidades.
3. `length()` descobre o tamanho.
4. `charAt(0)` acessa o primeiro caractere.
5. O texto não precisa ser alterado para consultar seu conteúdo.

Percorrendo a cadeia caractere por caractere

Exemplo:

Crie um programa em JAVA que conte quantas vogais e quantas não vogais tem em uma frase digitada pelo teclado.

StringBuffer: inserir, apagar e alterar caracteres

Quando a cadeia precisa ser modificada internamente, o livro utiliza StringBuffer, cujo conteúdo pode variar conforme a necessidade.

```
StringBuffer texto = new StringBuffer("Java");

texto.append(" Estruturado");
texto.insert(0, "Curso: ");
texto.setCharAt(0, 'C');
texto.delete(0, 7);
texto.reverse();

System.out.println(texto);
```

Métodos

append(...) → acrescenta
insert(pos, ...) → insere
delete(i, f) → apaga de i até f-1
setCharAt(i, c) → substitui caractere
reverse() → inverte a cadeia

Ao vivo: analisador de frase

10–12 min

PISTAS

Receba uma frase pelo teclado e mostre:

- a frase sem espaços no início e no fim;
- a quantidade de caracteres;
- a frase em maiúsculas;
- a quantidade de vogais;
- cada palavra em uma linha separada.

```
trim()  
length()  
toUpperCase()  
charAt(i)  
split(" ")
```

Arquivos: por que precisamos deles?

Cap. 11

Variáveis

Guardam dados enquanto o programa está em execução. Quando o programa termina, esses valores normalmente deixam de existir.

MEMÓRIA

Programa encerra → dados se perdem

Arquivos

Permitem armazenar dados de forma persistente. Em Java, o acesso pode ocorrer por bytes, caracteres ou objetos; nesta aula, o foco será arquivo texto.

DISCO

Programa encerra → arquivo permanece

Classe File: o caminho abstrato até o arquivo

A classe File associa um objeto a um arquivo ou diretório físico. Ela representa o caminho e permite verificações, mas não é a classe usada para ler ou gravar o conteúdo.

```
File arquivo = new File("dados.txt");  
  
System.out.println(arquivo.exists());  
System.out.println(arquivo.isFile());  
System.out.println(arquivo.canRead());  
System.out.println(arquivo.canWrite());  
System.out.println(arquivo.length());
```

Métodos úteis

exists() → existe fisicamente?
isFile() → é arquivo?
isDirectory() → é pasta?
canRead() → pode ser lido?
canWrite() → pode receber gravação?
length() → tamanho do arquivo

Manipulação de arquivos e IOException

O livro destaca que operações de abertura, leitura, gravação e fechamento podem gerar IOException. Por isso, os comandos de arquivo devem ser tratados com try-catch.

```
import java.io.*;

try {
    // abrir, ler ou gravar arquivo
}
catch (IOException e) {
    System.out.println("Erro ao manipular arquivo.");
}
```

O erro pode não depender da lógica do programa: arquivo indisponível, caminho inválido, permissão, falha de leitura, entre outros.

Gravando caracteres com FileWriter

FileWriter define objetos capazes de escrever caracteres em arquivos de texto. O segundo parâmetro true permite acrescentar novos dados sem destruir o conteúdo já existente.

```
File arquivo = new File("dados.txt");

try {
    FileWriter escritor = new FileWriter(arquivo, true);
    escritor.write("Olá, arquivo!\n");
    escritor.close();
}
catch (IOException e) {
    System.out.println("Erro ao manipular arquivo.");
}
```

Atenção

new FileWriter(arquivo)
→ sobrescreve o conteúdo.

new FileWriter(arquivo, true)
→ acrescenta ao final.

write(...)
→ grava caractere ou String.

close()
→ fecha o arquivo.

Lendo caracteres com FileReader

FileReader define objetos capazes de ler caracteres de um arquivo de texto. O método `read()` devolve o próximo caractere como número inteiro; quando retorna -1, o fim do arquivo foi alcançado.

```
File arquivo = new File("dados.txt");

try {
    FileReader leitor = new FileReader(arquivo);
    int valor;

    while ((valor = leitor.read()) != -1) {
        System.out.print((char) valor);
    }

    leitor.close();
} catch (IOException e) {
    System.out.println("Erro ao manipular arquivo.");
}
```

Fluxo

1. Criar File
2. Criar FileReader
3. Ler com `read()`
4. Converter para char
5. Repetir até -1
6. Fechar com `close()`

Fluxo mínimo: criar, gravar e ler

1 • CAMINHO

```
File arq = new  
File("dados.txt");
```

2 • GRAVAÇÃO

```
FileWriter esc =  
    new FileWriter(arq, true);  
esc.write("Java\n");  
esc.close();
```

3 • LEITURA

```
FileReader lei =  
    new FileReader(arq);  
int x;  
while ((x = lei.read()) != -1)  
    System.out.print((char)x);  
lei.close();
```

Projeto ao vivo: cadastro de alunos em arquivo texto

≈ 20 min

Objetivo Usar um arquivo como local de armazenamento de registros simples.

Cada aluno será gravado em uma linha:

```
nome;curso;nota1;nota2
```

Exemplo físico no arquivo:

```
Ana;COMPUTACAO;8.0;7.5  
Bruno;ADS;6.0;9.0
```

Etapas

1. Ler os dados pelo teclado.
2. Montar uma String de registro.
3. Gravar com `FileWriter(true)`.
4. Ler o arquivo com `FileReader`.
5. Separar linhas e campos.
6. Calcular e mostrar a média.

Cadastro de alunos — 1/4: entrada e normalização

```
Scanner entrada = new Scanner(System.in);
File arquivo = new File("alunos.txt");

System.out.print("Nome: ");
String nome = entrada.nextLine().trim();

System.out.print("Curso: ");
String curso = entrada.nextLine().trim().toUpperCase();

System.out.print("Nota 1: ");
double n1 = Double.parseDouble(entrada.nextLine());

System.out.print("Nota 2: ");
double n2 = Double.parseDouble(entrada.nextLine());
```

O que revisar?

`nextLine()`
→ lê a linha completa

`trim()`
→ retira espaços do início e do fim

`toUpperCase()`
→ converte para maiúsculo

`Double.parseDouble()`
→ converte texto em double

Cadastro de alunos — 2/4: montar e gravar o registro

```
try {  
    FileWriter escritor = new FileWriter(arquivo, true);  
  
    String registro = nome + ";" + curso + ";"  
        + n1 + ";" + n2 + "\n";  
  
    System.out.println("Registro montado:");  
    System.out.println(registro);  
  
    escritor.write(registro);  
    escritor.close();  
}  
catch (IOException e) {  
    System.out.println("Erro ao manipular arquivo.");  
}
```

A ideia do registro

Uma linha do arquivo reúne os campos separados por ;

Exemplo:

Ana;COMPUTACAO;8.0;7.5

FileWriter(..., true) preserva o conteúdo anterior e acrescenta o novo aluno ao final.

Cadastro de alunos — 3/4: ler todo o arquivo

```
FileReader leitor = new FileReader(arquivo);
StringBuffer conteudo = new StringBuffer();
int valor;

while ((valor = leitor.read()) != -1) {
    conteudo.append((char) valor);
}

leitor.close();

System.out.println("Conteúdo bruto:");
System.out.println(conteudo);
```

Fluxo de leitura

FileReader
→ lê caracteres

read()
→ devolve o próximo caractere

-1
→ indica fim do arquivo

StringBuffer
→ acumula os caracteres lidos

Cadastro de alunos — 4/4: separar campos e calcular

```
String[] linhas = conteudo.toString().split("\\R");

for (String linha : linhas) {
    if (linha.isBlank()) {
        continue;
    }

    String[] campos = linha.split(";");
    double nota1 = Double.parseDouble(campos[2]);
    double nota2 = Double.parseDouble(campos[3]);
    double media = (nota1 + nota2) / 2.0;

    System.out.println("Aluno: " + campos[0]);
    System.out.println("Curso: " + campos[1]);
    System.out.println("Média: " + media);
}
```

Integração

`split("\\R")`
→ separa as linhas

`split(";")`
→ separa os campos

O retorno é `String[]`:
voltamos a usar vetores.

`parseDouble()`
→ recupera as notas como
números.

Questões do livro-texto — cadeias de caracteres

- 1 Receba uma frase e mostre cada palavra em uma linha separada.
- 2 Receba uma frase e gere uma nova retirando os espaços excedentes no início, no fim e entre as palavras.
- 3 Receba uma frase e conte quantas palavras ela possui depois de corrigir os espaços excedentes.
- 4 Receba uma frase e conte quantos verbos existem nela, considerando que os verbos terminam em R.

Fonte: Capítulo 9 — exercícios propostos e exercício resolvido 25.

Questões do livro-texto — arquivos

- 1 Crie um arquivo de alunos em que cada registro tenha número, nome, curso, nota 1 e nota 2.
- 2 Inclua alunos no arquivo, evitando que dois alunos possuam o mesmo número.
- 3 Consulte o número, o nome e a média de todos os alunos cadastrados no arquivo.
- 4 Consulte os alunos aprovados, considerando média maior ou igual a 7.

Fonte: Capítulo 11 — exercícios resolvidos 1, 2, 6 e 7.

Fechamento: o que precisa ficar

Checklist técnico

- ☐ String representa uma cadeia de caracteres.
- ☐ length() e charAt() permitem percorrer o texto.
- ☐ equals() compara conteúdo.
- ☐ StringBuffer permite inserir, apagar e alterar.
- ☐ File representa o caminho do arquivo.
- ☐ FileWriter grava caracteres.
- ☐ FileReader lê caracteres.
- ☐ true no FileWriter permite acrescentar.
- ☐ read() retorna -1 no fim do arquivo.
- ☐ Operações de arquivo devem tratar IOException.

Livro-texto

ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi de. Fundamentos da programação de computadores: algoritmos, PASCAL, C/C++ (padrão ANSI) e JAVA. 3. ed. Pearson, 2012.

Capítulo 9 — Manipulando cadeias de caracteres.

Capítulo 11 — Arquivo.

Próximo passo: consolidar registros em Java e relacioná-los com persistência.